Bonus I: for Java Sumative Assessment–D. Ender

1. Explain How test-driven development applies to security: writing failing tests that encode security requirements first.
2. Explain the importance of writing Abuse-case tests.

1. How Test-Driven Development (TDD) Applies to Security

Test-driven development (TDD) applies to security by writing security requirements as tests before writing the code that implements them.

The basic process is:

1. Write a security test first that describes the expected secure behavior.
2. Run the test and confirm that it fails because the security feature has not been implemented yet.
3. Write the minimum code needed to make the test pass.
4. Refactor the code while ensuring the security tests continue to pass.

For example, if a program must reject invalid CVE IDs, you could first write a test such as:

> "The application must reject an input that does not match the required CVE format."

The test initially fails. After implementing the validation, the test passes.

This is valuable for security because it makes security requirements executable and repeatable rather than relying only on documentation or manual testing.

---

2. Importance of Abuse-Case Tests

Abuse-case tests test what happens when someone intentionally uses the system in an unexpected or malicious way.

Normal tests ask:

> "Does the system work when a legitimate user uses it correctly?"

Abuse-case tests ask:

> "What happens when an attacker tries to misuse it?"

Examples include testing whether the application:

- Rejects malformed or excessively long input.
- Prevents unauthorized access.
- Handles invalid authentication attempts safely.
- Rejects unexpected file paths or commands.
- Prevents duplicate or manipulated records.
- Handles extremely large or unexpected values without crashing.
- Does not expose sensitive information in error messages.

Abuse-case testing is important because security failures often occur at the boundaries of expected behavior. Testing malicious and abnormal inputs helps identify vulnerabilities before attackers can exploit them.

In short: TDD makes security requirements testable from the beginning, while abuse-case tests make sure the system remains secure when users or attackers deliberately try to break its assumptions.